

DATA, AUTHENTICATION & SECURITY

BACKEND REVIEW

How the Appwrite database, authentication, storage and payment flow are wired, where the weaknesses are, and what has already been closed.

SECURITY POSTURE

81/100

Findings	13 (1 critical, 3 high)
Resolved this pass	5
Collections	11
Server functions	37 across 8 modules

ARCHITECTURE

HOW THE DATA LAYER IS WIRED

The core design is sound, and it is the thing most projects at this size get wrong: the browser never touches the database. Appwrite collections carry no public permissions, and every read and write goes through a server function that verifies the caller's session JWT before using a server-only API key.

Database	Appwrite Databases, database id from APPWRITE_DATABASE_ID (default 'bakery')
Auth	Appwrite Account – email/password plus Google OAuth2 token flow
Storage	Appwrite Storage, bucket 'products', public read, 10MB cap, image types only
Server access	REST via server-only API key in src/integrations/appwrite/admin.server.ts
Client access	None. The browser SDK is used only for session management
Transport	TanStack Start server functions; every call carries a short-lived Appwrite JWT
Payments	Razorpay payment links, confirmed by a signed webhook
Email	Resend, server-side only

Collections

COLLECTION	HOLDS	ACCESS	NOTES
profiles	Customer name, phone, address, lat/lng	Owner via server fn	Document id is the Appwrite user id
user_roles	admin / customer grants	Admin only	Bootstrapped from APPWRITE_ADMIN_EMAILS
categories	Catalogue taxonomy	Public read	Seeded in code when empty
products	Catalogue, pricing, variants	Public read / admin write	Seeded in code when empty
orders	Order header, slot, totals, payment	Owner + admin	Ownership checked on every mutation
order_items	Line items	Owner + admin	Written only alongside an order
reviews	Product ratings	Verified purchasers	Gated on a completed order containing the product
blackout_dates	Closed days	Public read / admin write	
newsletter_subscribers	Email list	Admin read	Subscribe is public by design
newsletter_campaigns	Sent campaigns	Admin only	
offer_codes	Promo and voucher codes	Admin write / public validate	Falls back to an in-process list on failure

ACCESS CONTROL

ENDPOINT EXPOSURE MATRIX

Admin protection is applied consistently: all eleven admin endpoints call `requireAppwriteAuth` and then `assertAdmin` against the `user_roles` collection. Reviews are gated on a completed order that actually contained the product, and order mutations re-check ownership rather than trusting the client.

ENDPOINT	REQUIRES	ASSESSMENT
<code>getCatalog</code> , <code>getProductBySlug</code> , <code>getBlackoutDates</code>	Public	Correct – catalogue data is public
<code>getPublicOfferCodes</code>	Public	Correct – only active, visible codes are returned
<code>checkOfferCode</code>	Public	Acceptable, but unthrottled – see F-06
<code>subscribeToNewsletter</code>	Public	Correct by design
<code>unsubscribeFromNewsletter</code>	Public	Weak – see F-07
<code>getProductReviews</code>	Public	Correct
<code>claimGameRewardCoupon</code>	Authenticated	Fixed this pass – was public
<code>getMyProfile</code> , <code>saveMyProfile</code> , <code>placeOrder</code> , <code>getMyOrders</code>	Authenticated	Correct
<code>cancelRescheduledOrder</code> , <code>reportOrderIssue</code>	Authenticated + ownership	Correct
<code>getMyReviewState</code> , <code>submitReview</code>	Authenticated + purchase check	Correct
<code>getMyAccess</code>	Authenticated	Correct
All 11 <code>admin.functions</code> endpoints	Authenticated + <code>assertAdmin</code>	Correct and consistent
<code>saveSiteContent</code> , <code>resetSiteContent</code>	Authenticated + <code>assertAdmin</code>	Correct
Offer code admin endpoints	Authenticated + <code>assertAdmin</code>	Correct

Already correct before this review

- CSRF middleware is explicitly re-registered in `src/start.ts`, which TanStack Start would otherwise drop once that file exists.
- Zod validators run on every mutating endpoint.
- The webhook uses `timingSafeEqual` with a length check rather than string comparison.
- `.env` is untracked, and no server secret is reachable from client code.

- The Appwrite API key is confined to a server-only module that client code never imports.

FINDINGS

DETAILED FINDINGS

F-01 . Universal OTP backdoor and PII disclosure

CRITICAL

FIXED

`src/lib/auth.server.ts, src/lib/auth.functions.ts`

`verifyPhoneOtp` accepted the literal code "123456" for any number, so phone verification could be passed for a phone the caller did not control. `sendOtpToPhone` also returned the generated code to the caller as `devCode`. Both endpoints were unauthenticated and unthrottled, and the verify response returned the matching account's id, full name and home address – so the pair could be scripted to walk phone numbers and harvest customer addresses. The OTP store was an unbounded module-level Map, which neither survives nor is shared across workers.

Resolved • The entire phone-OTP surface was removed. No UI referenced it. `requestPasswordRecovery` went with it – a stub that reported "recovery email sent" without sending anything, while Appwrite's own `sendPasswordRecovery` already did the real work.

F-02 . Unauthenticated, unmetered discount-code minting

HIGH

FIXED

`src/lib/offers.functions.ts, src/lib/offers.server.ts`

`claimGameRewardCoupon` required no session and generated a fresh random suffix on every call, so the arcade endpoint could be scripted into unlimited live 15%-off codes.

Resolved • Claiming now requires a session, and the suffix is derived from `SHA-256(userId:gameId)` rather than `Math.random`. Replaying a game returns the voucher already held instead of minting another, capping each account at one voucher per game. Playing remains open to signed-out visitors.

F-03 . Orders recorded as paid at creation

HIGH

FIXED

`src/lib/orders.server.ts`

Every new order was written with `paid_at` set to the current time and a synthetic `payment_ref`, despite the status being `pending_approval` and no money having moved. Financial records stated a payment that had not happened.

Resolved • Payment fields stay null until the Razorpay webhook records them. The admin flow (approve, mint link, webhook confirms) is unchanged.

FINDINGS CONTINUED

F-04 · Payment webhook verified only the signature

HIGH

FIXED

`src/routes/api/public/razorpay-webhook.ts`

The handler checked the HMAC and then confirmed the order unconditionally. It did not check that the order existed, that it had not already been paid, that it had not been rejected or completed, or that the amount received covered the total. Razorpay retries, and a replayed body carries a valid signature.

Resolved · The handler now loads the order, returns early if it is already paid, refuses to revive rejected or completed orders, and compares the amount received (in paise) against the order total before confirming.

F-05 · No security response headers

MEDIUM

FIXED

`src/server.ts`

Responses carried no HSTS, no nosniff, no framing policy, no referrer policy, no permissions policy and no content security policy.

Resolved · All are now set on every response. HSTS, nosniff, X-Frame-Options/frame-ancestors, Referrer-Policy, Permissions-Policy and COOP are enforced. The CSP ships report-only until CSP_ENFORCE=true, so it can be validated against real traffic before it is able to break a live page.

F-06 · No rate limiting on any public endpoint

MEDIUM

OPEN

`All public server functions`

`checkOfferCode`, `subscribeToNewsletter` and the catalogue endpoints accept unlimited requests. Promo codes can be brute-forced, and the subscribe endpoint can be used to flood the list.

Recommended · Add a token bucket in front of the public endpoints keyed on CF-Connecting-IP, backed by Cloudflare KV or a Durable Object so the count survives across workers. Ten attempts per minute is generous for code validation.

FINDINGS CONTINUED

F-07 . Newsletter unsubscribe takes any address

MEDIUM

OPEN

`src/lib/newsletter.server.ts`

`removeSubscriber` unsubscribes whatever email it is given, with no proof the caller controls it. Anyone can unsubscribe anyone.

Recommended • Issue a signed, expiring token in the unsubscribe link and require it. An HMAC of the email with a server secret is enough and needs no extra storage.

F-08 . Stock is never checked or decremented

MEDIUM

OPEN

`src/lib/orders.server.ts`

`createOrderForUser` checks `is_active` but never reads stock, and no path reduces it. Any quantity of any active product can be ordered regardless of what the admin recorded.

Recommended • Check available stock while pricing the lines and reject with a per-product message. Decrement in the same pass. Left open deliberately: applying it against live data without knowing the current stock values could start rejecting valid orders.

F-09 . Promo redemption is not atomic

MEDIUM

OPEN

`src/lib/offers.server.ts`

`validatePromoCode` reads `used_count` and `markOfferCodeUsed` writes it back in a separate request. Two orders placed at once both pass validation, so a single-use voucher can be redeemed twice. `markOfferCodeUsed` failures are also swallowed, so the count can silently not move at all.

Recommended • Guard the increment with a conditional write on the value just read and retry on conflict, or move redemption into an Appwrite Function that owns the document.

FINDINGS CONTINUED

F-10 • Silent in-memory fallback for offer codes

MEDIUM

OPEN

`src/lib/offers.server.ts`

Every offer-code write is wrapped in a try/catch that falls back to a module-level array. In a serverless deployment that array is per-instance and lost on recycle, so a failed write reports success and the code later does not exist. It also masks genuine outages.

Recommended • Keep the seed list for local development only, and let write failures propagate in production so the caller sees the error.

F-11 • Unused Supabase credentials ship to the browser

LOW

OPEN

`src/integrations/supabase/, src/integrations/lovable/index.ts, .env`

No application code imports the Supabase client; its only importer is the Lovable-generated `integrations/lovable/index.ts`, which is itself unreferenced. Despite that, the Supabase client and the `VITE_SUPABASE_*` values are present in the shipped client bundle, adding weight and publishing configuration for a backend the site does not use.

Recommended • The generated file is marked "auto-generated by Lovable. Do not modify", so it was left in place rather than deleted. Confirm with Lovable that the integration can be removed, then drop the folder, the `@supabase/supabase-js` dependency and the `SUPABASE_*` environment variables.

F-12 • Admin bootstrap is email-based and permanent

LOW

OPEN

`src/lib/roles.server.ts`

Any account signing in with an address listed in `APPWRITE_ADMIN_EMAILS` is granted admin on first sign-in, and the grant is never re-checked or revoked. If that mailbox is ever compromised or the address reused, admin follows it.

Recommended • Treat the bootstrap as first-run only: once any admin exists, stop auto-granting and require an existing admin to promote. Add role revocation to the admin surface.

FINDINGS CONTINUED

F-13 . Appwrite errors are returned to the client verbatim

LOW

OPEN

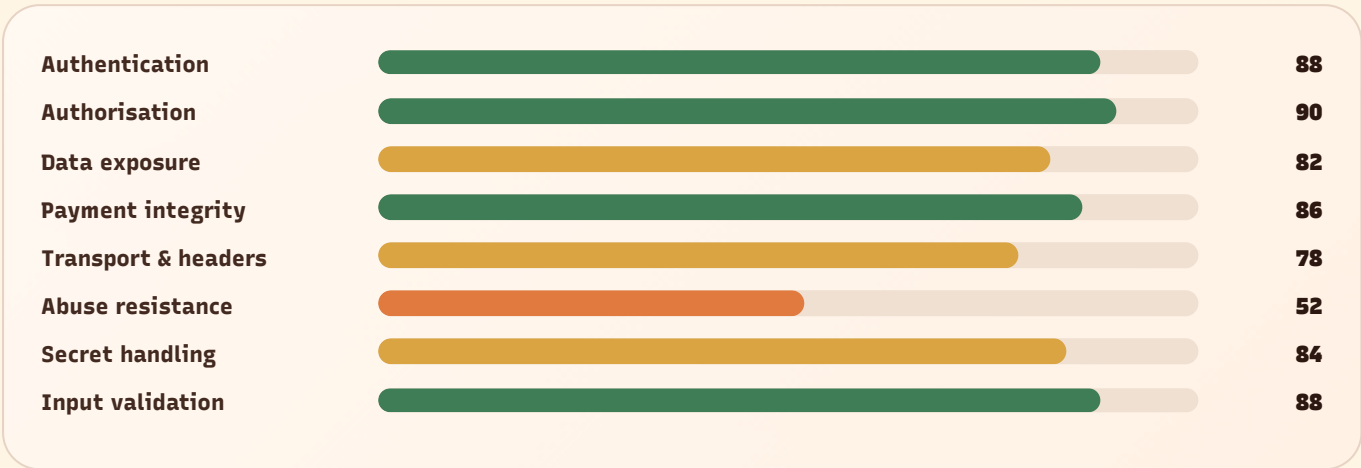
`src/integrations/appwrite/admin.server.ts`

`request()` throws ``Appwrite ${status}: ${text}``, and those messages reach the browser through server-function errors, exposing collection names, attribute names and internal validation detail.

Recommended • Log the full error server-side and return a generic message with a correlation id.

POSTURE

WHERE THE RISK SITS NOW



Authentication	Appwrite-managed sessions, JWT-verified server side, OAuth supported
Authorisation	Every admin endpoint gated identically; ownership checked on order mutations
Data exposure	PII disclosure closed; verbose upstream errors remain
Payment integrity	Signature, amount, state and replay now all verified
Transport & headers	Full header set applied; CSP still report-only by choice
Abuse resistance	No rate limiting anywhere; promo redemption not atomic
Secret handling	.env untracked, no server secrets client-side, unused Supabase keys bundled
Input validation	Zod on every mutating endpoint

The one number to watch

Abuse resistance scores 52 because there is no rate limiting anywhere in the application. Every other category is in good shape, so this is now the single largest gap: promo codes can be brute-forced and public endpoints can be flooded. On Cloudflare this is a small change – a token bucket in KV keyed on CF-Connecting-IP – and it would lift this category above 80 on its own.

WHAT TO DO NEXT

HARDENING ROADMAP

NOW

- Set CSP_ENFORCE=true after reviewing report-only violations for a few days.
- Rate-limit the public endpoints on CF-Connecting-IP via KV or a Durable Object.
- Sign unsubscribe links so an address cannot be removed by a third party.
- Make promo redemption atomic with a conditional write and retry.

NEXT

- Enforce and decrement stock during order creation.
- Stop the silent in-memory fallback for offer-code writes in production.
- Replace verbatim Appwrite errors with generic messages plus a correlation id.
- Close the admin bootstrap once the first admin exists, and add revocation.

LATER

- Remove the unused Supabase integration once Lovable confirms it is safe.
- Move script-src off 'unsafe-inline' by adopting nonces for the streamed hydration scripts.
- Add an audit log for admin actions – status changes, price edits, code creation.
- Schedule Appwrite API key rotation and scope the key down to the collections actually used.

Operational checklist

ITEM	STATUS
.env excluded from version control	CONFIRMED
Server secrets absent from the client bundle	CONFIRMED
CSRF protection on server functions	CONFIRMED
Webhook secret configured (RAZORPAY_WEBHOOK_SECRET)	VERIFY IN PRODUCTION
CSP moved from report-only to enforced	PENDING REVIEW
Appwrite API key scoped to the collections in use	TO REVIEW

Key rotation schedule agreed

TO AGREE

Backup and restore rehearsed for
the bakery database

TO REHEARSE